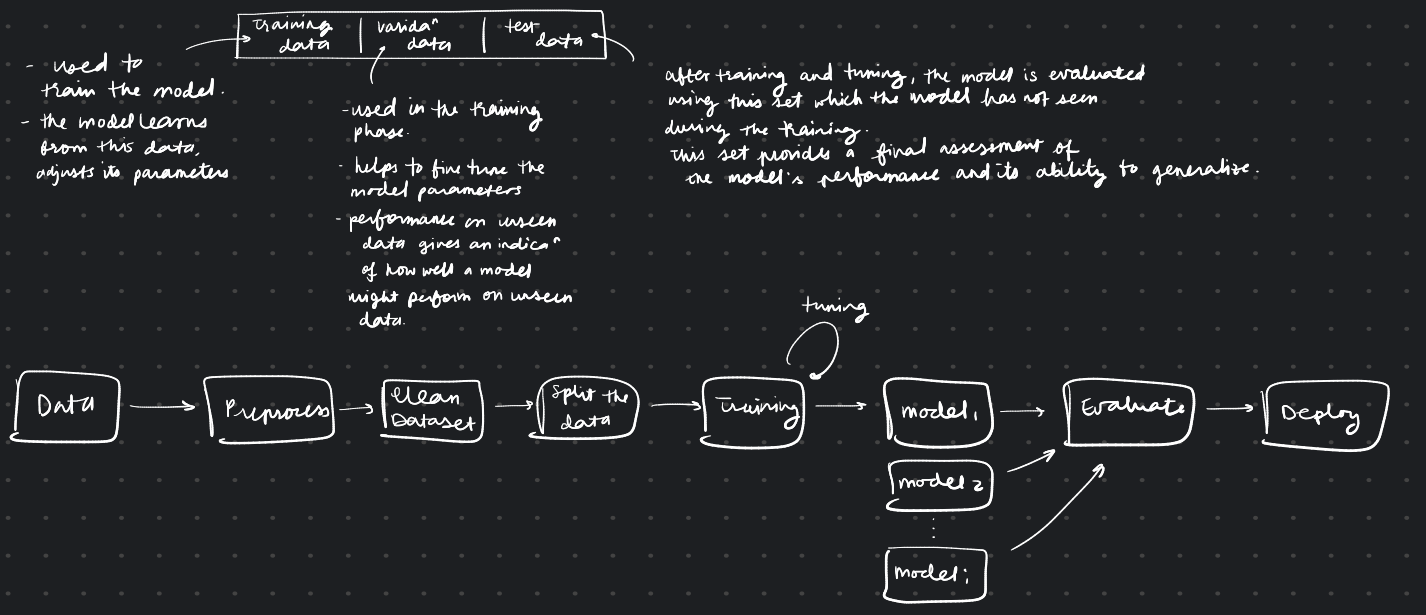
॥श्रीगणेशाय नमः॥

Model Evaluation and Validation

Model evaluation

- It is a crucial step in the model development process. It allows us to assess how well a model performs on unseen data.
  A model's effectiveness is not only determined by its performance on the training data set, but also by its ability to generalize.
- Generalization: The goal of ML is to develop models that perform well on new, unseen data. Evaluation metrics help us quantify this performance.
- Avoid overfitting: Evaluating models helps to identify whether a model is too complex (overfitting) or too simple (underfitting).
- Model selection: Evaluation allows for the comparison of different models to choose the best one for a given task.

Model evaluation process



Challenges in model evaluation

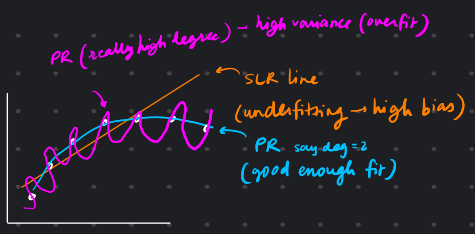
- Data leakage: Ensuring that no information from the test set is used during training or validation is crucial to maintain the integrity of evaluation.
- Imbalanced datasets: In cases where classes are not equally represented, traditional metrics (like accuracy) may not provide a true picture of model performance. Techniques such as stratified sampling or specialized metrics (like F1-score) become important.
- Computational efficiency: For very large datasets or complex models, evaluation can be time consuming. Approaches such as cross-validation may be used to efficiently utilize the data.

Bias-variance tradeoff

Bias

The error due to overly simplistic assumptions in the model.
High bias means the model is too simplistic to capture underlying patterns of the data.

$$\text{Bias} = E[\hat{y}] - y$$



Addressing bias:
- Increase model complexity: Use more complex models that capture the underlying relationships in the data.
- Use more features: By incorporating more features,

Variance

The error due to the model's sensitivity to small fluctuations in the training data. High variance means the model is too complex, capturing noise in the data, leading to overfitting.

Addressing variance:
- Simplifying the model: Reduce the model complexity (pruning, reducing degree of polynomial, etc.), using fewer features, or using simpler models.
- Regularization: Applying techniques like ridge and lasso regression to penalize large coefficients helps in reducing the complexity of the model.
- Increasing train data: More data helps in generalizing better.
- Ensembling methods: Techniques such as bagging and boosting combine multiple models to reduce variance.
- Cross validation: Methods like k-fold cross validation and LOO cross validation to assess model's performance on multiple subsets of training data helps reducing overfitting.

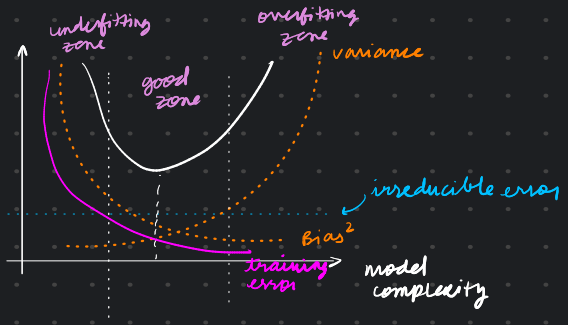
Bias-variance tradeoff

The total error can be broken down into 3 components.

Total error = Bias^2 + variance + irreducible error

- Total error: how far off the model's predictions are from the actual data.
- variance: how much the predictions fluctuate with the dataset.
- irreducible error: due to inherent noise in the data; cannot be avoided;

Tradeoff:
- Bias ↑ variance ↓ (Too simple)
- Bias ↓ variance ↑ (Too complex)



The best fit will be given by the hypothesis on the trade-off point.

Cross validation

CV

- Statistical method used to estimate the skill of an ML model.
- It involves partitioning the dataset into subsets, training the model on some subsets and validating it on some subsets.
- This process is generally repeated multiple times to obtain an average model performance metric.

Advantages:
- Model robustness (generalization)
- Reduces overfitting and variance
- Hyperparameter tuning
- Reliable model comparison

Disadvantages:
- Computationally expensive.
- Time consuming.
- Choice of the technique.

Holdout validation

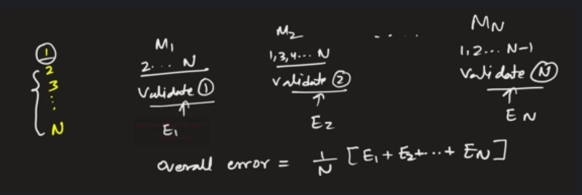
- Simplest method for estimating a model's performance by splitting the data into 3 separate parts: train, validation, and test sets.
- The model is trained on the training set and evaluated using the validation set if further training is required. Then finally, it is tested on the test set.

Advantages:
- Simple and quick.
- Performance estimation.

Disadvantages:
- High variance.
- Risk of bias.
- Not suitable for small datasets.

Leave-one-out cross validation (LOOCV)

- Each data point in the data set serves as a validation point once, while the remaining points form the training set. This is repeated for each data point.



- For a dataset with n points, $\{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$

In i^{th} iteration, the model is trained on $\{(x_1, y_1), \ldots, (x_{i-1}, y_{i-1}), (x_{i+1}, y_{i+1}), \ldots, (x_n, y_n)\}$

- The validation error E_i for this iteration is calculated as the difference between the actual and the predicted values for x_i.

$$\text{overall LOOCV error} = \frac{1}{N} \sum_{i=1}^{N} E_i$$

k-fold cross validation

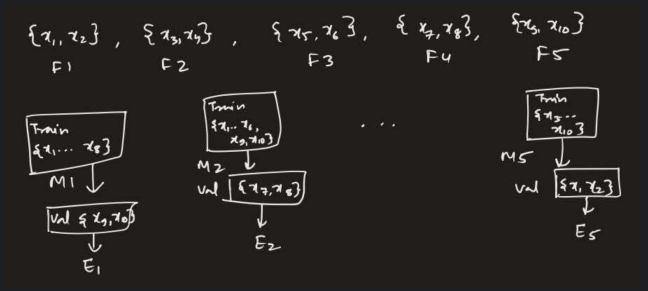
- The dataset is divided into k equal-sized subsets, or 'folds'. The model is trained k times, leaving out one fold as the validation set and using the remaining (k-1) folds as the training set.

1. Split the dataset into k equally-sized parts (folds).
2. For each fold:
   i. Train the model on remaining (k-1) folds.
   ii. Validate the model on this fold.
3. Average the performance metric across all k iterations to get the overall performance estimate. $E_{CV} = \frac{1}{k}\sum_{i=1}^{k} E_i$

The folds will be mutually exclusive.

Common choices of k are 5 or 10. The choice of k depends on the dataset size and computational resources.
N-fold CV == LOOCV

eg: We have 10 data points $\{x_1, \dots x_{10}\}$, we want a regression model to be be evaluated using 5-Fold cross validation.



Overall k-Fold CV error = $\frac{1}{5}[E_1 + E_2 + \dots + E_5]$.

eg: Suppose we have the following data set with single feature inputs and regression outputs.

	x	y
①	1	2
②	2	3
③	3	2.5
④	4	5

Calculate the 2-Fold CV error metric when training a SLR model on this dataset. The folds are {①, ②}, {③, ④}

→ 1) Train on fold {①, ②}, validate on {③, ④}:

(1,2), (2,3) $y = mx + c$

$m = \frac{3-2}{2-1} = \frac{1}{1}$ $\quad 3 = 2 \cdot 1 + c$, $c = 1$ $\quad y = x + 1$

$m = \frac{\sum x_i y_i - \sum x_i \sum y_i}{N\sum x_i^2 - (\sum x_i)^2}$ $\quad c = \frac{\sum y_i - m\sum x_i}{N}$

validn: (3, 2.5) $\hat{y} = 3 + 1 = 4$

$(\hat{y} - y)^2 = (4 - 2.5)^2 = 1.5^2$

(4, 5) $\hat{y} = 5$, $y = 5$

$(\hat{y} - y)^2 = 0^2 = 0$

$E_1 = \frac{1}{2}[1.5^2 + 0^2] = 1.125$

2) Train on fold {③, ④}.

(3, 2.5), (4, 5)

$m = \frac{5 - 2.5}{4 - 3} = 2.5$ $\quad 5 = 2.5(4) + c$, $c = -5$

$y = 2.5x - 5$

Test on {①, ②}

(1,2): $\hat{y} = (2.5)(1) - 5 = -2.5$

$(y - \hat{y})^2 = (2 + 2.5)^2 = 4.5^2$

(2,3): $\hat{y} = (2.5)(2) - 5 = 0$

$(y - \hat{y})^2 = (0 - 3)^2 = 3^2$

$E_2 = \frac{1}{2}[4.5^2 + 3^2] = 14.625$

$E = \frac{1.125 + 14.625}{2} = 7.875$ ← result of 2-fold CV.

Advantages:
- Less bias: Since each observation is used for both training and validation, k-folds is a reliable metric for CV.
- Less variance: Averaging the results across k folds makes the metric more stable.

LOO	K- folds	Holdout
$N \times (N-1)$	$K \times (N - \frac{N}{K})$	$1 \times (N \times 0.8)$
models D_{train}	models D_{train}	model D_{train}

Disadvantages:
- Computationally expensive.
- Choice of k.

Stratified cross validation

- For many classification problems, especially those with imbalanced data, the class distribution is not uniform.
  E.g. 10% data in the minority class, 90% in the majority.

{B, B, B, B, ...} {A, A, ...}
Train set validation set

- Holdout CV or regular k-folds CV might create folds where the minority class(es) are underrepresented in the training data. This can leave to misleading performance estimators.
- Stratified sampling:

1000 < 40% F / 60% m

Normal sampling for 10 data points
{4F, 6M}
{5F, 5M}
{1F, 9M}
↓
Any of these might happen

stratified sampling
{4F, 6M}
Class proportion is maintained in the samples.

Stratified k-folds CV:
1. Split the dataset into k folds, maintaining the class distribution in each fold (stratified sampling). ← only change
2. For each fold:
   i. Train the model on remaining (k-1) folds.
   ii. Validate the model on this fold.
3. Average the performance metric across all k iterations to get the overall performance estimate.

The folds need not be mutually exclusive.

Advantages:
- Better performance on unbalanced data.

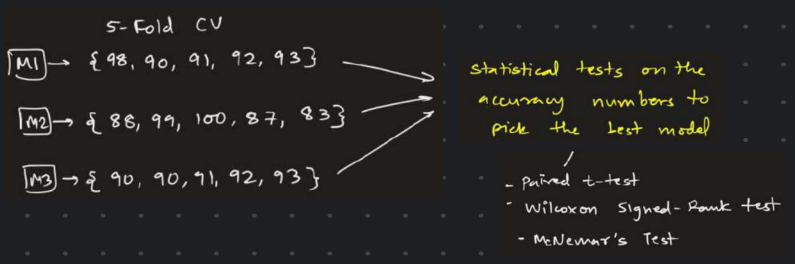
Model selection

The process of choosing the best model among several candidates based on the performance on validation data. Often, we need to compare multiple models with different configurations, architectures, or hyperparameters to select that one which best generalizes on unseen data.

- Prevents overfitting or underfitting by choosing a model that generalizes well.
- Optimizes performance by selecting the model that achieves the best balance between accuracy and complexity.
- Reduces computational cost by avoiding more complex models when simpler models are sufficient.

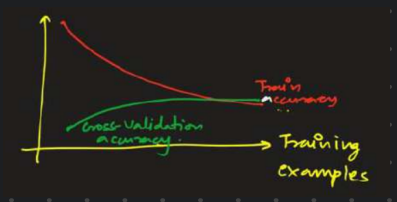
Criteria for model selection

1. Performance metrics: Select a model with better accuracy metrics.
2. Complexity: Models with fewer parameters are typically less prone to overfitting and are easier to interpret.
3. Training time and resources.
4. Stability: Choose a model which is less sensitive to variations to data.



Common pitfalls

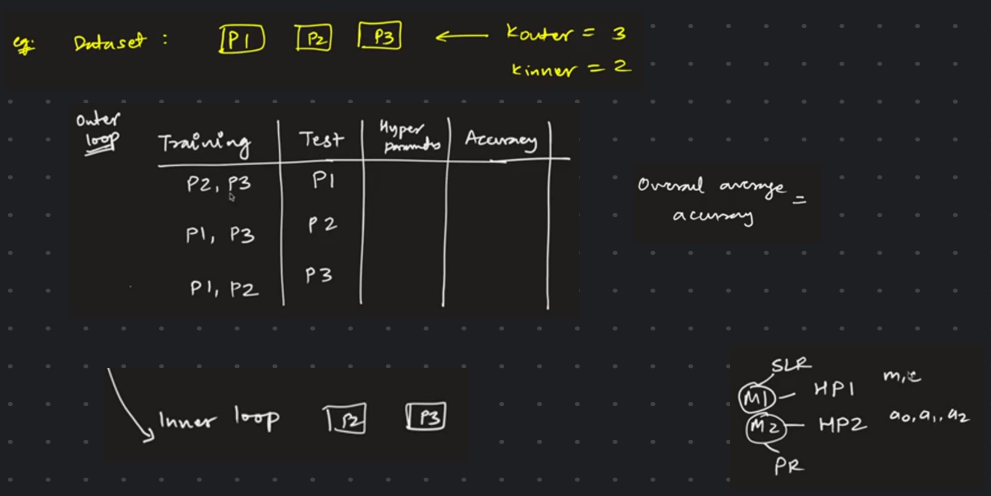
- Overfitting on validation set.
- Ignoring model complexity.
- Over-reliance on a single metric.

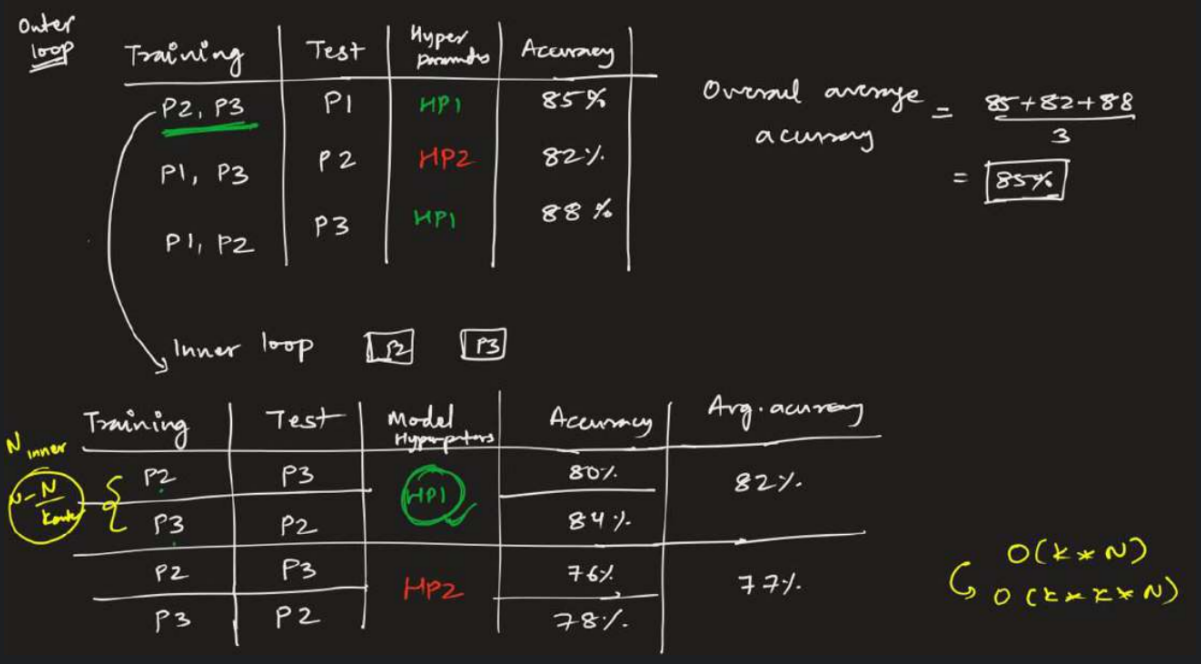


Nested cross validation

Two levels of CV:
- Outer loop (outer CV): Splits the dataset into training and testing sets.
- Inner loop (inner CV): Performs hyperparameter tuning within the training set only.

1. Determine the # of folds.
   - Choose k_outer folds for the outer loop.
   - Choose k_inner folds for the inner loop.
2. Split the dataset D into k_outer folds.
3. For each outer fold:
   - Reserve one fold as the outer test set.
   - Use the remaining (k_outer - 1) folds as the training set in the inner loop.
4. Perform k_inner-fold CV on (k_outer - 1) folds training data.
   Use the inner loop to tune hyperparameters based on the inner validation folds.
5. Use the best hyperparameters from the inner loop to train the model on the entire outer training set.
   Evaluate the model on the outer test set and record the performance metric.
6. Repeat the process for each outer fold and average the performance metrics from each outer test set to obtain an unbiased estimate.





$$K_{outer} * K_{inner} * \left(N_{inner} - \frac{N_{inner}}{K_{inner}}\right) + K_{outer} * N_{inner}$$

N_{inner}: size of dataset for inner loop; N_{inner}: size of dataset for outer loop

$$N_{inner} = N - \frac{N}{K_{outer}}$$

Disadvantage: Computationally expensive than k-folds CV

Advantage :- less bias & variance than k-folds CV or other similar algorithms.